

1.1 INTRODUCCIÓN A LA PROGRAMACIÓN

Una computadora es una máquina y, como otras máquinas, como un automóvil o una podadora, debe encenderse y luego conducirse, o controlarse, para hacer la tarea que se pretende realizar. En un automóvil, por ejemplo, el control es proporcionado por el conductor, quien se sienta en su interior y lo dirige. En una computadora, el conductor es un conjunto de instrucciones llamado programa. De manera más formal, un **programa de computadora** es un conjunto independiente de instrucciones usado para operar una computadora con el fin de producir un resultado específico. Otro término para un programa o conjunto de programas es **software**, y se usarán ambos términos de manera indistinta a través del texto.¹

El proceso de escribir un programa, o software, se llama **programación**, mientras al conjunto que puede usarse para construir un programa se llama **lenguaje de programación**. Los lenguajes de programación disponibles se presentan en una variedad de formas y tipos.

Lenguaje de máquina

En su nivel más fundamental, los únicos programas que pueden usarse en realidad para operar una computadora son los **programas en lenguaje de máquina**. Tales programas, los cuales también se conocen como **programas ejecutables**, o **ejecutables** para abreviar, consisten en una secuencia de instrucciones compuestas por números binarios como:²

```
11000000 000000000001 000000000010
11110000 000000000010 000000000011
```

Estas instrucciones en lenguaje de máquina constan de dos partes: una de instrucción y una de dirección. La parte de instrucción, a la cual se conoce como **opcode** (abreviatura de “código de operación”), por lo general es el conjunto de bits en el extremo izquierdo de la instrucción y le indica a la computadora la operación a realizar, como sumar, restar, multiplicar, etc., mientras los bits en el extremo derecho especifican las direcciones de memoria de los datos que se van a usar. Por ejemplo, suponiendo que los ocho bits en el extremo izquierdo de la primera instrucción enlistada antes contienen el código de operación para sumar, y los siguientes dos grupos de doce bits son las direcciones de los dos operandos que se van a sumar, esta instrucción sería un comando para “sumar los datos en la ubicación 1 de la memoria a los datos en la ubicación 2 de la memoria”. Del mismo modo, suponiendo que el opcode 11110000 significa multiplicar, la siguiente instrucción es un comando para “multiplicar los datos en la ubicación 2 de la memoria por los datos en la ubicación 3”. (La sección 1.6 explica cómo convertir de números binarios a decimales.)

Lenguajes ensambladores

Debido a que cada clase de computadora, como IBM, Apple y Hewlett Packard, tiene su propio lenguaje de máquina particular, es muy tedioso y tardado escribir esos programas en lenguaje de máquina.³ Uno de los primeros avances en la programación fue la sustitución de

¹De una manera más incluyente, el término software también se usa para denotar tanto los programas como los datos con los que operarán los programas.

²Revise la sección 1.6 al final de este capítulo si no está familiarizado con los números binarios.

³En la actualidad, el lenguaje en el nivel de máquina está definido por el procesador alrededor del cual está construida la computadora.

símbolos en forma de palabras, como ADD, SUB, MUL, por los opcodes binarios y los números decimales y etiquetas por las direcciones en memoria. Por ejemplo, usando estos símbolos y valores decimales para las direcciones en memoria, las dos instrucciones en lenguaje de máquina anteriores pueden escribirse como:

ADD 1, 2

MUL 2, 3

Los lenguajes de programación que usan este tipo de notación simbólica se conocen como **lenguajes ensambladores**. Debido a que las computadoras sólo pueden ejecutar programas en lenguaje de máquina, el conjunto de instrucciones contenido dentro de un programa en lenguaje ensamblador debe traducirse a un programa de lenguaje de máquina antes que pueda ejecutarse en una computadora. Los programas traductores que realizan esta función para los programas en lenguaje ensamblador se conocen como **ensambladores** (véase la figura 1.1).

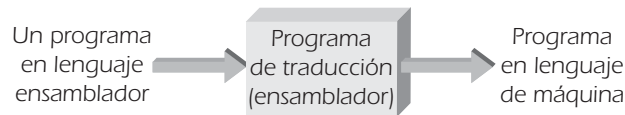


Figura 1.1 Los programas ensambladores deben traducirse.

Lenguajes de niveles bajo y alto

Tanto los lenguajes de máquina como los ensambladores se clasifican como **lenguajes de nivel bajo**. Esto se debe a que ambos tipos de lenguaje usan instrucciones que se vinculan en forma directa con un tipo de computadora. Como tal, un programa en lenguaje ensamblador está limitado porque sólo puede usarse con el tipo de computadora específica para el cual se escribió. Sin embargo, estos programas permiten usar las características especiales de un tipo de computadora particular y por lo general se ejecutan en el nivel más rápido posible.

En contraste con los lenguajes de nivel bajo están los lenguajes de alto nivel. Un **lenguaje de alto nivel** usa instrucciones que se parecen a los lenguajes escritos, como el inglés, y pueden ejecutarse en una variedad de tipos de computadora. Visual Basic, C, C++ y Java son ejemplos de lenguajes de alto nivel. Usando C++, una instrucción para sumar dos números y multiplicarlos por un tercer número puede escribirse como:

```
resultado = (primero + segundo) * tercero;
```

Los programas escritos en un lenguaje de computadora (de alto o bajo nivel) se conocen como **programas fuente** y **código fuente**. Una vez que se ha escrito un programa en un lenguaje de alto nivel también debe traducirse, como un programa ensamblador de bajo nivel, al lenguaje de máquina de la computadora en que se va a ejecutar. Esta traducción puede lograrse en dos formas.

Cuando cada declaración en un programa fuente de alto nivel es traducida de manera individual y ejecutada inmediatamente después de la traducción, el lenguaje de programación usado se llama **lenguaje interpretado** y el programa que hace la traducción se llama **intérprete**.

Cuando todas las instrucciones en un programa fuente de alto nivel son traducidas como una unidad completa antes que cualquier declaración sea ejecutada, el lenguaje de programa-

ción usado se llama **lenguaje compilado**. En este caso, el programa que hace la traducción se llama **compilador**. Pueden existir tanto versiones compiladas como interpretadas de un lenguaje, aunque de manera típica predomina una. C++ es predominantemente un lenguaje compilado.

La figura 1.2 ilustra la relación entre un código fuente de C++ y su compilación en un programa ejecutable en lenguaje de máquina. Como se muestra, el programa fuente se introduce usando un programa editor. Éste es en efecto un programa procesador de palabras que es parte del ambiente de desarrollo proporcionado por el compilador. Debe entenderse, sin embargo, que la introducción del código sólo puede comenzar después que una aplicación se ha analizado y comprendido en forma minuciosa y el diseño del programa ha sido planeado con cuidado. La forma en que se logra esto se explica en la siguiente sección.

La traducción del programa fuente C++ en un programa en lenguaje de máquina comienza con el compilador. La salida producida por el compilador se llama **programa objeto**, el cual es una versión en lenguaje de máquina del código fuente. En casi todos los casos, su código fuente usará código preprogramado existente, con código que ha escrito con anterioridad o código proporcionado por el compilador. Éste podría incluir código matemático para encontrar una raíz cuadrada, por ejemplo, o código que se está reutilizando de otra aplicación. Además, un programa C++ grande puede almacenarse en dos o más archivos de programa separados. En todos estos casos, este código adicional debe combinarse con el programa objeto antes que el programa pueda ejecutarse. Es tarea del **ligador** lograr este paso. El resultado del proceso de ligamiento es un programa en lenguaje de máquina completado, que contiene todo el código requerido por el programa, el cual está listo para su ejecución. El último paso en el proceso es cargar este programa en lenguaje de máquina en la memoria principal de su computadora para su ejecución real.

Orientaciones a procedimientos y a objetos

Además de clasificar los lenguajes de programación como de alto o bajo nivel, también se clasifican por su orientación a procedimientos u objetos. En un **lenguaje orientado a procedimientos** las instrucciones disponibles se usan para crear unidades independientes, conocidas como **procedimientos**. El propósito de un procedimiento es aceptar datos como entrada y transformarlos de alguna manera para producir un resultado específico como una salida. Hasta la década de los años 90 la mayor parte de los lenguajes de programación de alto nivel eran orientados a procedimientos.

En la actualidad, un segundo enfoque, la orientación a objetos, ha tomado el escenario central. Una de las motivaciones para **lenguajes orientados a objetos** fue el desarrollo de pantallas gráficas y soporte para las interfaces gráficas de usuario (GUI), capaces de desplegar múltiples ventanas que contienen tanto formas gráficas como texto. En tal ambiente, cada ventana en la pantalla puede considerarse un objeto con características asociadas, como color, posición y tamaño. Usando un enfoque orientado a objetos, un programa debe definir primero los objetos que manipulará, incluyendo una descripción de las características generales de los objetos y unidades específicas para manipularlos, como cambiar el tamaño y la posición y transferir datos entre objetos. Es de igual importancia que los lenguajes orientados a objetos tiendan a soportar la reutilización del código existente con más facilidad, lo cual elimina la necesidad de revalidar y reexaminar código nuevo o modificado. C++, el cual se clasifica como un lenguaje orientado a objetos, contiene características que se encuentran en los lenguajes orientados a procedimientos y a objetos. En este texto se diseñarán, desarrollarán y presentarán ambos tipos de código, que es la forma en que se escribe la mayor parte de los programas C++ actuales. Debido a que el código C++ orientado a objetos siempre contiene algún código

de procedimientos, y muchos programas C++ simples se escriben por completo usando sólo código de procedimientos, este tipo de código se presenta primero.

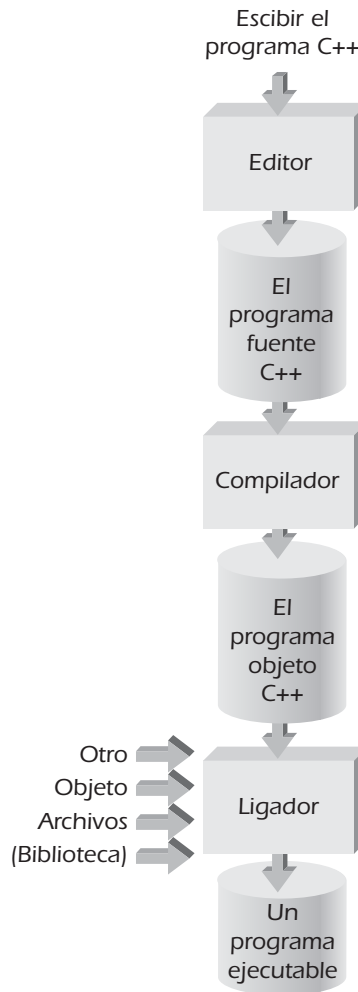


Figura 1.2 Creación de un programa C++ ejecutable.

Software de aplicación y de sistema

El software de aplicación y el software de sistema son dos categorías lógicas de programas de computadora. El **software de aplicación** consiste en aquellos programas escritos para realizar tareas particulares requeridas por los usuarios. Todos los programas en este libro son ejemplos de software de aplicación.

El **software de sistema** es la colección de programas que deben estar disponibles en cualquier sistema de cómputo en el que ha de operar. En los primeros entornos de cómputo de las décadas de los años 50 y 60, el usuario tenía que cargar al inicio el software de sistema en forma manual para preparar la computadora para que hiciera algo. Esto se llevaba a cabo usando hileras de conmutadores en un panel frontal. Se decía que aquellos comandos iniciales

introducidos en forma manual iniciaban (**boot**) la computadora, una expresión derivada de la expresión inglesa *pulling oneself up by the bootstraps* que significa “salir adelante sin ayuda”. En la actualidad, el llamado cargador inicial (**bootstrap loader**) es un componente permanente que se ejecuta de manera automática desde el software del sistema de la computadora.

De manera colectiva, el conjunto de programas de sistema usados para operar y controlar una computadora se llama **sistema operativo**. Los sistemas operativos modernos incluyen las siguientes tareas: administración de memoria; asignación de tiempo de CPU; control de unidades de entrada y salida como teclado, pantalla e impresoras, y la administración de todos los dispositivos de almacenamiento secundarios. Muchos sistemas operativos manejan programas grandes y múltiples usuarios, en forma concurrente, dividiendo los programas en segmentos que son movidos entre el disco y la memoria conforme se necesita. Tales sistemas operativos permiten que más de un usuario ejecute un programa en la computadora, lo cual le da a cada usuario la impresión que la computadora y los periféricos son sólo suyos. Esto se conoce como un sistema **multiusuario**. Además, muchos sistemas operativos, incluyendo la mayor parte de los ambientes con ventanas, permiten a cada usuario ejecutar múltiples programas. Dichos sistemas operativos se conocen como sistemas **multiprogramados** y **multitareas**.

El desarrollo de C++

En un nivel básico, el propósito de casi todos los programas de aplicación es procesar datos para producir uno o más resultados específicos. En un lenguaje de procedimientos, un programa se construye a partir de conjuntos de instrucciones, con cada conjunto nombrado como un procedimiento, como se señaló con anterioridad. En efecto, cada procedimiento mueve los datos un paso más cerca de la salida final deseada a lo largo de la ruta mostrada en la figura 1.3.

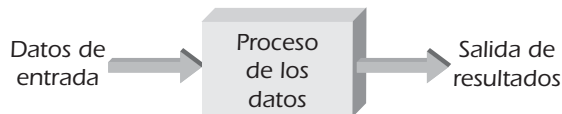


Figura 1.3 Operaciones de procedimiento básicas.

El proceso de programación ilustrado en la figura 1.3 refleja en forma directa las unidades de hardware de entrada, procesamiento y salida usadas para construir una computadora (véase la sección 1.6). Esto no fue accidental porque los primeros lenguajes de programación fueron diseñados de manera específica para corresponder y controlar en forma directa, lo más óptimamente posible, a las unidades de hardware apropiadas.

El primer lenguaje de procedimientos, llamado FORTRAN, cuyo nombre se deriva de *FOR*mula *TRAN*slation, fue introducido en 1957 y siguió siendo popular durante la década de los años 60 y principios de la década de los años 70. (Otro lenguaje de programación de nivel alto desarrollado en forma casi concurrente con FORTRAN, pero que nunca logró la aceptación abrumadora de FORTRAN, fue nombrado ALGOL.) FORTRAN tiene instrucciones tipo álgebra que se concentran en la fase de procesamiento mostrada en la figura 1.3 y fue desarrollado para aplicaciones científicas y de ingeniería que requerían salidas numéricas de gran precisión, incluyendo muchos lugares decimales. Por ejemplo, calcular la trayectoria de un cohete o el nivel de concentración bacteriana en un estanque contaminado, como se ilustra en la figura 1.4, requiere evaluar una ecuación matemática a un alto grado de precisión numérica y es típico de las aplicaciones basadas en FORTRAN.

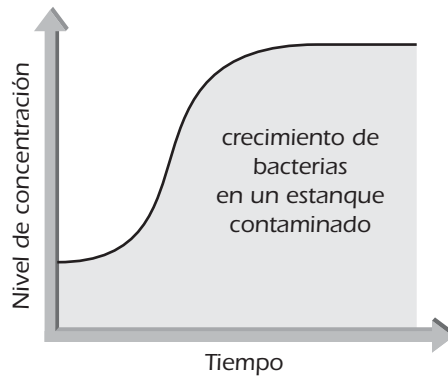


Figure 1.4 FORTRAN fue desarrollado para aplicaciones científicas y de ingeniería.

En orden de aparición, el siguiente lenguaje de aplicación de nivel alto significativo fue COBOL, el cual fue introducido en la década de los años 60 y permaneció como un lenguaje de procedimientos importante hasta la década de los años 80. La palabra COBOL se formó por las siglas de Common Business-Oriented Language. Este lenguaje tenía características enfocadas hacia aplicaciones de negocios que requerían cálculos matemáticos más simples que los necesarios para aplicaciones de ingeniería. Uno de los beneficios más notables de COBOL fue que proporcionaba formatos de salida extensos que facilitaban la creación de informes que contenían muchas columnas de números y totales en dólares y centavos formateados con esmero, como se ilustra en la figura 1.5. Esto obligó a los programadores a construir procedimientos estructurados bien definidos que seguían un patrón más consistente que el requerido por FORTRAN.

No. de parte	Descripción	Cantidad	Precio
12225	#4 Clavos, normales	25 cajas	1.09
12226	#6 Clavos, normales	30 cajas	1.09
12227	#8 Clavos, normales	65 cajas	1.29
12228	#10 Clavos, normales	57 cajas	1.35
12229	#12 Clavos, normales	42 cajas	
12230	#16 Clavos normales		

Figure 1.5 COBOL fue desarrollado para aplicaciones de negocios.

Otro lenguaje, BASIC (o Beginners All-purpose Symbolic Instruction Code), fue desarrollado en Dartmouth College más o menos al mismo tiempo que COBOL. BASIC era en esencia una versión ligeramente reducida de FORTRAN y pretendía ser un lenguaje introductorio para estudiantes universitarios. Era un lenguaje relativamente sencillo, fácil de entender, que no requería un conocimiento detallado de una aplicación específica. Su principal desventaja era que no requería ni imponía un enfoque consistente o estructurado para crear programas. Con frecuencia, el programador no podía comprender con facilidad qué hacía su programa BASIC después de un tiempo breve.

Para remediar esto y adecuar la programación a una base más científica y racional que hiciera más fácil entender y reutilizar el código, se desarrolló el lenguaje Pascal. (Pascal no es una sigla, sino que se le puso este nombre en honor al matemático del siglo xvii Blaise Pascal.) Introducido en 1971, proporcionó a los estudiantes un fundamento más firme en el diseño de programación estructurada que lo aportado por versiones anteriores de BASIC.

Los programas estructurados se crean usando un conjunto de estructuras bien definidas organizadas en secciones de programación individuales, cada una de las cuales ejecuta una tarea específica que puede probarse y modificarse sin perturbar otras secciones del programa. Sin embargo, el lenguaje Pascal estaba estructurado en forma tan rígida que no existían escapes de las secciones estructuradas cuando hubieran sido útiles. Esto era una limitante para muchos proyectos del mundo real y es una de las razones por las que Pascal no fue aceptado en forma amplia en los campos científico y de ingeniería. En cambio, el lenguaje C, el cual es un lenguaje de procedimientos estructurado desarrollado en la década de los años 70 en AT&T Bell Laboratories por Ken Thompson, Dennis Ritchie y Brian Kernighan, se convirtió en el lenguaje para aplicaciones de ingeniería dominante de la década de los años 80. Este lenguaje tiene un amplio conjunto de capacidades que permite que se escriba como un lenguaje de nivel alto mientras conserva la capacidad de acceso directo a las características del nivel de máquina de una computadora.

C++ fue desarrollado a principios de la década de los años 80, cuando Bjarne Stroustrup (también en AT&T) usó sus conocimientos en lenguaje de simulación para crear un lenguaje de programación orientado a objetos. Una característica central de los lenguajes de simulación es que modelan situaciones de la vida real como objetos. Esta orientación a objetos, la cual era ideal para objetos gráficos presentados en pantalla como rectángulos y círculos, se combinó con características de C, existentes para formar el lenguaje C++. Por tanto, C++ conservó el conjunto extenso de capacidades estructuradas y de procedimientos proporcionadas por C, pero agregó su propia orientación a objetos para convertirse en un verdadero lenguaje de programación de uso general. Como tal, C++ puede usarse desde programas interactivos simples, hasta programas de ingeniería y científicos sofisticados y complejos, dentro del contexto de una estructura en verdad orientada a objetos.

Ejercicios 1.1

1. Defina los siguientes términos:
 - a. programa de computadora
 - b. programación
 - c. lenguaje de programación
 - d. lenguaje de alto nivel
 - e. lenguaje de bajo nivel
 - f. lenguaje de máquina
 - g. lenguaje ensamblador
 - h. lenguaje orientado a procedimientos
 - i. lenguaje orientado a objetos
 - j. programa fuente
 - k. compilador
 - l. intérprete
2. Describa el propósito y usos principales del software de aplicación y de sistema.

3. a. Describa la diferencia entre lenguajes de alto y bajo nivel.
b. Describa la diferencia entre lenguajes orientados a procedimientos y a objetos.
4. Describa las semejanzas y diferencias entre ensambladores, intérpretes y compiladores.
5. a. Dados los siguientes códigos de operación,

11000000 significa sumar el 1er. operando al 2o. operando
 10100000 significa restar el 1er. operando del 2o. operando
 11110000 significa multiplicar el 2o. operando por el 1er. operando
 11010000 significa dividir el 2o. operando entre el 1er. operando

traduzca las siguientes instrucciones al español:

opcode	Dirección del 1er. operando	Dirección del 2o. operando
11000000	000000000001	0000000000010
11110000	000000000010	0000000000011
10100000	000000000100	0000000000011
11010000	000000000101	0000000000011

- b. Suponiendo que las siguientes ubicaciones contienen los datos proporcionados, determine el resultado producido por las instrucciones listadas en el ejercicio 5a. Para este ejercicio, suponga que cada instrucción es ejecutada de manera independiente de cualquier instrucción.

Dirección	Valor inicial (en decimales) almacenado en esta dirección
000000000001	5
000000000010	3
000000000011	6
000000000100	14
000000000101	4

6. Reescriba las instrucciones en el nivel de máquina enlistadas en el ejercicio 5a usando notación de lenguaje ensamblador. Use los nombres simbólicos ADD, SUB, MUL y DIV para operaciones de adición, sustracción, multiplicación y división, respectivamente. Al escribir las instrucciones use valores decimales para las direcciones.

1.2

SOLUCIÓN DE PROBLEMAS Y DESARROLLO DE SOFTWARE

Sin importar cuál campo de trabajo elija o cuál pueda ser su estilo de vida, tendrá que resolver problemas. Muchos de éstos, como sumar el cambio en su bolsillo, pueden resolverse rápido y fácil. Otros, como montar en bicicleta, requieren algo de práctica pero pronto se vuelven automáticos. Otros más requieren de una planeación y premeditación considerables para que la solución sea apropiada y eficiente. Por ejemplo, construir una red telefónica celular o crear un sistema de administración de inventarios para un gran almacén son problemas para los cuales las soluciones por ensayo y error podrían resultar costosas y desastrosas.